pfi

Python for Fixed Income

COURSE 2 · BOND MATH

Course 2 Capstone

WEEK 3 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The scenario: month end, a new extract

You are the analyst on the credit desk. Operations has sent the holdings extract for 30 November 2026, with a cover note.

The portfolio manager wants

A book you have recomputed yourself

A risk report in Excel

One page: the book's risk, and what was wrong

Bond-math problems are planted in the file. Nothing names them: the rules and your module find them.

```
To: Credit desk
From: Operations
Subject: Month-end holdings extract, as of
2026-11-30

The month-end extract is attached as
course2_capstone_holdings.csv. It is the same
layout as last month.

Control totals for the book:
  Number of bonds:      199
  Total face:           493,000,000 dollars
  Total market value:   496,372,835.00 dollars

Market value is face times clean price plus
accrued interest, divided by 100.
All positions settle and are priced as of
2026-11-30. Analytics are semiannual, 30/360.
Please tell us if the extract does not agree with
these totals.
```

What you hand in: eight deliverables

#	Deliverable	Section
1	bondmath.py complete, python -m pytest -q passing, output saved	1
2	clean_book(raw, benchmark, curve) in capstone.py, with asserts	2 to 5
3	Reconciliation: recomputed against delivered, every difference explained	6
4	risk_report(book, ratings), written to a workbook of three sheets	7
5	The same risk_report on the September book, side by side	8
6	One Git commit per deliverable, and git log --oneline	every section
7	An AI-use log	9
8	A written description of the book's risk and of the data	9

clean_book recomputes price, accrued, yield, modified duration, DV01, convexity, spread, spread duration, and DTS.

The files, all invented

File	What it is
`course2_capstone_holdings.csv`	the extract, as of 2026-11-30: Course 1's 19 columns
`course2_capstone_cover_note.txt`	control totals and conventions, read by the setup
`benchmark.csv`	the September benchmark, as of 2026-09-30
`course2_treasury_curve.csv`	the invented curve of day 11, at ten tenors
`ratings.csv`	the rating scale: bucket and grade
`holdings.csv`	the September book, clean (days 7 to 14)

Coupon and yield in percent. OAS in bp. Durations in years. DTS in years x bp. Price and accrued per 100. Face and value in dollars.

Nine sections, three sittings

First sitting

1 Module, first commit

2 Test the extract

3 Fix the inputs

Days 1 to 8, Course 1's project

Second sitting

4 Reprice every bond

5 Package it: clean_book

6 The reconciliation

Days 6 to 13

Third sitting

7 The risk report

8 September, November

9 Log and description

Days 10 to 14

Then the closing sections of every day: Excel Side by Side, Save the Day with Git, Bring It Home, and the AI check.

Testing the extract: the desk's rules

Rule	What the desk expects of an extract
1	One row for each bond held. No row is a copy of another
2	The maturity is after the as-of date
3	No number is missing
4	Coupon and yield in percent, price and accrued per 100 of face
5	Market value = face x (price + accrued) / 100, to the cent
6	Every analytic reproduces from the terms with bondmath, within tolerance
7	Identifiers, ratings, and sizes are well formed

Section 2 finds

Count on the book as it arrived. Fix nothing yet.

Section 3 fixes inputs

One decision at a time: evidence, then headline() before and after.

Matching the control totals is necessary, and it is not enough.

Rule 6: reprice every bond

Column of the file	Recomputed with	Tolerance
price, accrued, duration, spread_duration	price, accrued_interest, ...	0.0005 + 1e-9
yield_pct	yield_to_maturity	0.001 percent
oas	spread_to_curve_bp	1e-9 bp
dts	dts	0.05 + 1e-9

1 Recompute it for every bond **2 Count the rows outside the tolerance** **3 What do those rows share?** **4 Decide, then fix the book**

Settlement is each row's as-of date; semiannual, 30/360, as the cover note says. Two questions are about two bonds you find yourself.

How the checks work

A check holds a short code, never the answer

```
check('Question 2.1', answer_2_1, 'a short code')
```

```
correct
```

```
not yet. You have ...
```

```
not attempted yet
```

Whole numbers as 7 or 7.0. Decimals to the places asked. Text ignores capitals.

Closed blocks and two checkpoints

"Stuck after trying?" holds the decision the checkpoint expects for one kind of problem. Not a count, not an answer. Leave it closed until you have decided.

Checkpoints after section 4 and after clean_book: every line correct before you go on.

A different decision can be defensible. Say so, and expect the checks to differ.

clean_book, then the reconciliation

```
%%writefile capstone.py
def clean_book(raw, benchmark, curve):
    """The decisions, in order ..."""
    # work in a copy; settlement = as_of_date
    # each decision of sections 3 and 4
    # every analytic with bondmath
    # assert every rule, with a message
    return book

cleaned = capstone.clean_book(raw, benchmark, curve)
assert len(cleaned) == expected_bonds, "..."
```

Every difference gets a reason, in order

1 A decision of clean_book

2 Final coupon period

3 Rounding in the file, shown

4 Unexplained: the target is zero

Rules inside the function; control totals asserted after the call.

The risk report, then September

`positions`

One row per bond: weight, DV01 in dollars, DTS contribution

`summary`

The book, each sector, each rating bucket: DV01 summed; duration, spread, DTS weighted

`scenarios`

Rates -100, -25, +25, +100 bp; spreads +10 percent and +25 bp: full reprice and estimate

Every scenario is a mechanical what-if: yields move at once, settlement does not. None is a forecast.

Rerun, do not redo: the same clean_book and risk_report, unchanged, on the September book, then the two months side by side.

Git: one commit per deliverable

Commit	Files
Deliverable 1	.gitignore, bondmath.py, test_bondmath.py, the reference files
Deliverable 2	capstone.py with clean_book
Deliverable 3	reconciliation.csv
Deliverable 4	capstone.py with risk_report, and the workbook
Deliverable 5	book_comparison.csv
Deliverables 7, 8	ai_use_log.md, then description.md

```
Every command:  !git -C "{work}" ...   Before deliverable 4:  git diff --stat
```

git log --oneline is deliverable 6. Then git tag course2.

AI help, the log, and the AI check

Using an assistant

The docstring is the prompt. Read the code, predict, then test it against Excel or a control total.

The log, question 9.1

What you asked, what you accepted or rejected, how you checked it. "none used" is valid.

Never into an AI tool

Client names, positions, anything from your employer, credentials. Your firm's policy first.

The AI check: one step of clean_book, its docstring, and the code that came back. Test first from Excel's rows.

Code that runs is not the same as code that is right.

The written description

1 The book in numbers, each with its unit

2 Where the risk sits: sector and rating

3 What each scenario does to market value

4 What changed from September

5 The data: found, fixed, evidence, and clean

Describe. Never recommend.

No bond, issuer, or sector is attractive; nothing says what to do or what will happen. A scenario is a what-if, not a view.

300 to 450 words, written to description.md and committed. A number in almost every sentence.

What you can do now

- Price any bullet on any date, and solve its yield, as Excel does
- Accrued, duration, DV01, convexity, spread, spread duration, DTS
- Every function tested against Excel, every change kept in Git
- Recompute and reconcile a book that cannot be trusted as delivered
- A risk report in Excel, and a page that describes, never recommends

Course 3 starts from your my_bondmath folder. Keep it up to date, and bring it with you.